

Complex data types

structs, linked lists, enums, unions, typedef

struct

Example

Student management database

- name
- ID

Example

Student management database

- name
- ID
- Course
- Year
- Group

Example

Student management database

- name
- ID
- Course
- Year
- Group
- Subjects taken

Example

Student management database

- name: char *
- ID: int
- Course: char * or int
- Year: int
- Group: int
- Subjects taken: int * or char **

Then we define *k parallel arrays*, one for each student field.

Very complicated.

Example

Student management database

- name: char *
- ID: int
- Course: char * or int
- Year: int
- Group: int
- Subjects taken: int * or char **

How would we do this in Java?

Then we define *k parallel arrays*, one for each student field.

Very complicated.

In Java

```
class Student{
    String name;
    int ID;
    String course;
    int year;
    int group;
    Vector<String> subjects;
    ...
    public static void main(String args[]){
        Vector<Student> students;
        ...
    }
};
```


C struct: a complex data type

Student management database

Like a Java class, without methods

```
struct student{  
    char *name;  
    int ID;  
    char *course;  
    int year;  
    int group;  
    char **subjects;  
};
```

member
fields

```
class Student{  
    String name;  
    int ID;  
    String course;  
    int year;  
    int group;  
    Vector<String> subjects;  
};
```

C

Java

C struct: defining a type and declaring a variable

The code below is only for defining the *type*. There is no variable yet.

```
struct student{  
    char *name;  
    int ID;  
    char *course;  
    int year;  
    int group;  
    char **subjects;  
};
```

`struct student mike;` ← Declaring a variable of type `struct student`
`struct student mike[5];` ← Declaring an array of `struct student`

C struct: defining a type and declaring a variable

Type definition and variable declaration can also be merged:

```
struct student{  
    char *name;  
    int ID;  
    char *course;  
    int year;  
    int group;  
    char **subjects;  
} john;
```

```
struct student mike;
```

Accessing **struct** member fields

```
struct student {  
    char *name;  
    int id;  
};  
struct student mike;
```

```
printf("%s\n", mike.name);  
printf("%d\n", mike.id);
```

If a struct variable is a scalar (statically allocated) the internal fields are allocated using the “.” (dot) operator.

Just like in Java.

Accessing **struct** member fields

```
struct student {  
    char *name;  
    int id;  
};  
struct student *mikeptr;  
  
....  
printf("%s\n",  
    *mikeptr.name);
```

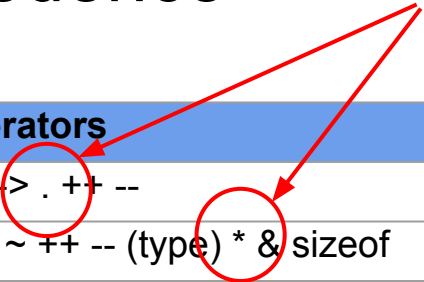
If a struct variable is a pointer...

Intuition says to dereference

?

Back to operator precedence

Operator “.” has precedence over dereference (*)



Precedence	Type	Operators	Associativity
1	Postfix	() [] .> . ++ --	Left to right
2	Unary	+ - ! ~ ++ -- (type) * & sizeof	Right to left
3	Multiplicative	* / %	Left to right
4	Additive	+ -	Left to right
5	Shift	<< >>	Left to right

```
printf("%s\n", *mikeptr.name);
```

Considering precedence, this is equivalent to

```
printf("%s\n", *(mikeptr.name));
```

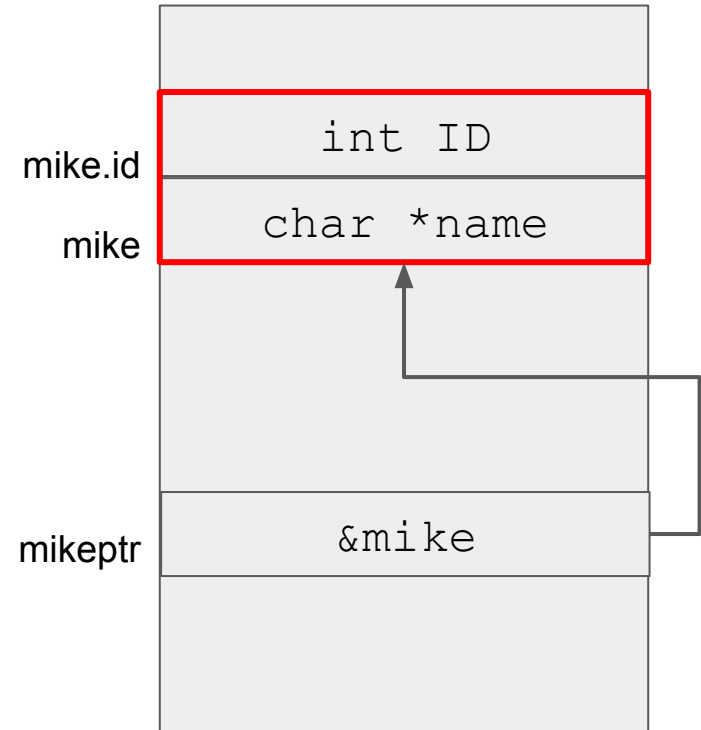
```

struct student {
    char *name;
    int id;
};
struct student mike;
struct student *mikeptr;
....
mikeptr = &mike;
printf("%s\n",
        *(mikeptr.name) );

```

mikeptr contains a memory address, not a struct.

Trying to access field “.name” would be an error (unspecified behaviour).



Accessing **struct** member fields

```
struct student {  
    char *name;  
    int id;  
};  
struct student *mikeptr;  
  
....  
printf("%s\n",  
        (*mikeptr).name);
```

If a struct variable is a pointer...

Intuition says to dereference.

- First dereference the pointer (`mikeptr`) so we get to the place in memory where the struct is
- Then access the field

Accessing **struct** member fields

```
struct student {  
    char *name;  
    int id;  
};  
struct student *mikeptr;  
  
....  
printf("%s\n",  
        (*mikeptr).name) ;  
printf("%s\n",  
        mikeptr->name) ;
```

If a struct variable is a pointer...

Intuition says to dereference.

- First dereference the pointer (`mikeptr`) so we get to the place in memory where the struct is
- Then access the field

This is a common operation $\Rightarrow$ there is a special operator that does this

- the “**->**” (arrow) operator.

Initialising structs: field by field

```
struct student {  
    char *name;  
    int id;  
};  
struct student mike;  
char mikesname[20];  
  
mike.id = 10;  
mike.name = mikesname;  
scanf("%s", mike.name);
```

Initialising structs: struct constants

```
struct student {  
    char *name;  
    int id;  
};  
struct student mike = {"Mike", 1};
```

Like an array initializer.
One constant for each field of the struct.

Initialising structs: *designated initialisers*

```
struct student {  
    char *name;  
    int id;  
};  
  
struct student mike = {  
    .name = "Mike",  
};
```

What if we don't want to initialise all the fields?

We can specify which fields should be initialised.

Fields that are not explicitly initialised are initialised according to their scope.

Initialising structs: assignment

```
struct student {  
    char *name;  
    int id;  
};  
struct student mike = {  
    .name = "Mike",  
};
```

```
struct student john;  
john = mike;
```

Even though a struct is a complex type it does support assignment.

Contrast this with arrays which do not support assignment.

Structs and functions

```
void increment(int value) {  
    value ++;  
}
```

```
void main(void) {  
    int number = 10;  
    increment(number);  
    printf("%d\n", number);  
}
```

Structs and functions

```
void increment(int value){  
    value ++;  
}
```

```
void main(void){  
    int number = 10;  
    increment(number);  
    printf("%d\n", number);  
}
```

```
> ./test
```

```
10
```

```
# Pass by value!
```

Structs and functions

```
void increment(int value[]){  
    value[0] ++;  
}
```

```
void main(void){  
    int array[] = {10,21};  
    increment(array);  
    printf("%d\n",array[0]);  
}
```


Structs and functions

```
void increment(int value[]){  
    value[0] ++;  
}
```

```
void main(void){  
    int array[] = {10,21};  
    increment(array);  
    printf("%d\n",array[0]);  
}
```

```
> ./test  
11
```

Pass by reference because the value of "array" is the address of the first array element



Structs and functions

```
struct value{
    int v;
};

void increment(struct value val){
    val.v++;
}

void main(void){
    struct value val_struct = {10};
    increment(val_struct);
    printf("%d\n", val_struct.v);
}
```

Structs and functions

```
struct value{
    int v;
};

void increment(struct value val){
    val.v++;
}

void main(void){
    struct value val_struct = {10};
    increment(val_struct);
    printf("%d\n", val_struct.v);
}
```

```
> ./test
10

# Pass by value!
```

Structs and functions

Structs behave as simple types.

They are always **passed by value** to functions.

This means that changes made to a struct within functions are lost when the function exits.

It also means that a large amount of information (the whole contents of the struct) are copied into the function's memory space

- If the struct is too large should use instead pointer to struct as function parameter

Structs and functions

```
int *new_and_init(void) {  
    int array[10];  
    ...  
    return array;  
}
```

```
void main(void) {  
    int *array = new();  
}
```

Structs and functions

```
int *new_and_init(void) {  
    int array[10];  
    ...  
    return array;  
}  
  
void main(void) {  
    int *array = new();  
}
```

Compiler warning that the function returns the address of a local variable.

The array is allocated memory statically within the function on the stack.

That memory will be lost when the function exits and reclaimed for a subsequent function call.

Structs and functions

```
struct value new_and_init(void) {  
    struct value v;  
    ...  
    return v;  
}
```

```
void main(void) {  
    struct value v;  
    v = new_and_init();  
}
```

Structs and functions

Structs behave as simple types.

When they are returned from a function it is their value that is returned, not just their address.

Structs as values, arrays as addresses

```
#include <stdio.h>

struct number{
    int n;
};

void main(void){
    struct number v = {0x10};
    int array[] = {0x10};

    printf("Array element = %x\n", array[0]);
    printf("Array value = %x\n", array);
    printf("Struct member = %x\n", v.n);
    printf("Struct value = %x\n", v);
}
```

```
> ./test
Array element = 10
Array value =
```

Structs as values, arrays as addresses

```
#include <stdio.h>

struct number{
    int n;
};

void main(void){
    struct number v = {0x10};
    int array[] = {0x10};

    printf("Array element = %x\n", array[0]);
    printf("Array value = %x\n", array);
    printf("Struct member = %x\n", v.n);
    printf("Struct value = %x\n", v);
}
```

```
> ./test
Array element = 10
Array value = 69272ac8
Struct member = 10
Struct value =
```

Structs as values, arrays as addresses

```
#include <stdio.h>

struct number{
    int n;
};

void main(void){
    struct number v = {0x10};
    int array[] = {0x10};

    printf("Array element = %x\n", array[0]);
    printf("Array value = %x\n", array);
    printf("Struct member = %x\n", v.n);
    printf("Struct value = %x\n", v);
}
```

```
> ./test
Array element = 10
Array value = 69272ac8
Struct member = 10
Struct value = 10
```

Structs as values, arrays as addresses

When we declare an array

- The value of the array itself is the address of the first element

When we declare a struct

- The value of the struct itself is the value of the first member of the struct

In this way structs behave in the same way as basic types (int, char, etc).

Allocating memory for structs

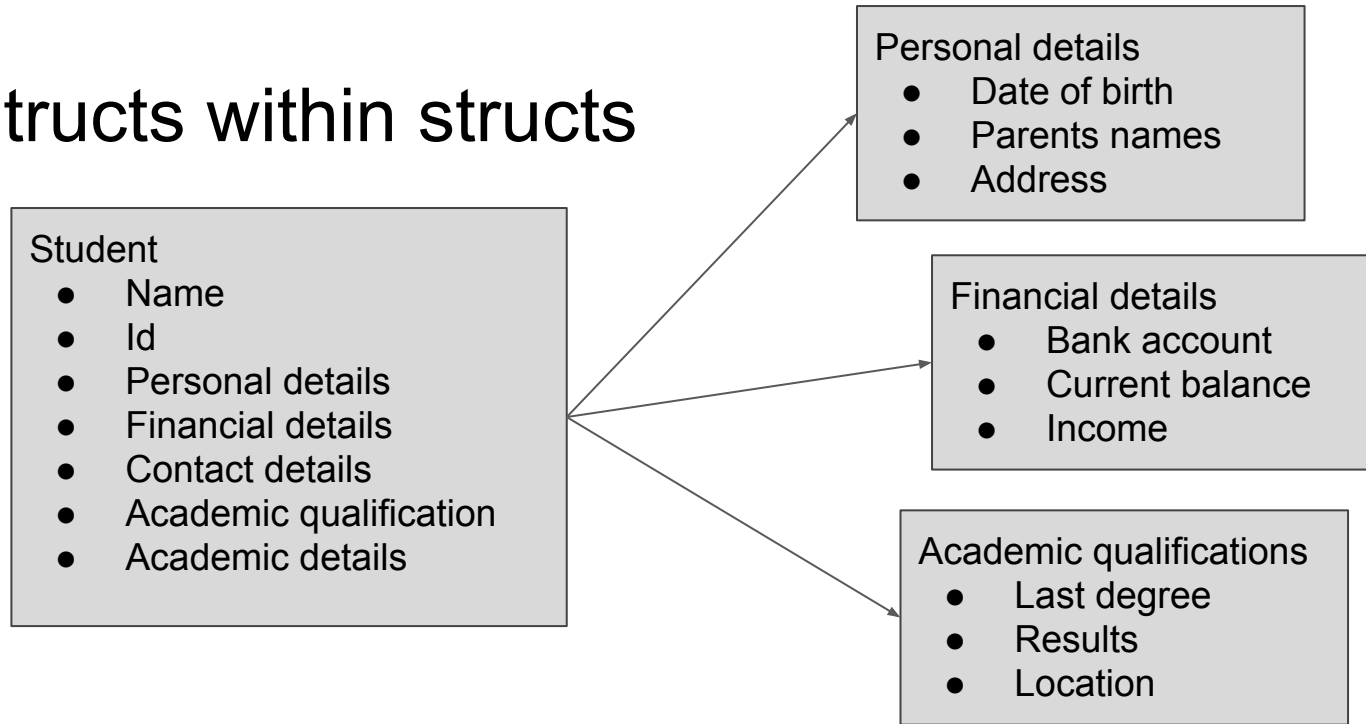
```
struct student {  
    char *name;  
    int id;  
};  
struct student mike;
```

sizeof operator should be used when using malloc.

It will determine how much memory the struct takes.

```
mike = malloc(sizeof(struct student));  
mike->id = 1;  
scanf("%s", mike->name);
```

Structs within structs



In complex applications we will have structures that are logically linked to other structures. If the relation is 1:1 then we can include the linked structures inside the main one.

Structs within structs

```
struct student{  
    char *name;  
    int ID;  
    struct _financial {  
        long bank_account;  
        int balance;  
    } financial;  
    struct _academic {  
        int year;  
        int group;  
        struct subject *subjects;  
    } academic;  
};
```

Looks like Java inner classes

Structs within structs: initialisation

```
struct student{
    char *name;
    int ID;
    struct _financial {
        long bank_account;
        int balance;
    } financial;
    struct _academic {
        int year;
        int group;
        struct subject *subjects;
    } academic;
};

struct student mike = {
    "Mike", 1,
    12345678, 2000,
    2020, 1, NULL
};
```

The diagram illustrates the mapping between the fields of the `struct student` and the values provided in the initialization of `mike`. Brackets on the left group the fields into their respective sub-structs: `_financial` (containing `bank_account` and `balance`) and `_academic` (containing `year`, `group`, and `subjects`). Arrows on the right point from these groups to the corresponding values in the initialization: `12345678` and `2000` for the `_financial` sub-struct, and `2020`, `1`, and `NULL` for the `_academic` sub-struct.

Structs within structs: access

```
struct student{
    char *name;
    int ID;
    struct _financial {
        Long bank_account;
        Int balance;
    } financial;
    struct _academic {
        int year;
        int group;
        struct subject *subjects;
    } academic;
};
```

```
struct student mike = {
    "Mike", 1,
    12345678, 2000,
    2020, 1, NULL
};

printf("Balance is %d\n",
    mike.financial.balance);
```

Structs within structs

Unlike Java inner classes,
Inner structs are accessible at global scope

```
struct student{
    char *name;
    int ID;
    struct _financial {
        Long bank_account;
        Int balance;
    } financial;
    struct _academic {
        int year;
        int group;
        struct subject *subjects;
    } academic;
};
```

```
struct student mike = {
    .name="Mike", .ID=1
};
struct _financial example =
{1234567, 2000};

mike.financial = example;

printf("Balance is %d\n",
    mike.financial.balance);

printf("Example balance: %d",
    example.balance);
```

Structs within structs

It is more common for a struct to contain a *pointer* to the other structs.

```
struct financial {  
    long bank_account;  
    int balance;  
};
```

```
struct student{  
    char *name;  
    int ID;  
    struct financial *financial;  
    struct academic *academic;  
};
```

```
struct student mike = {  
    .name="Mike", .ID=1  
};
```

Two arrows originate from the text 'Use a designated initialiser...' below. One arrow points to the '.name="Mike"' part of the struct initialization, and the other points to the '.ID=1' part.

Use a designated initialiser because we don't want to initialise the pointers in the struct.

Structs within structs

It is more common for a struct to contain a *pointer* to the other structs.

```
struct financial {  
    long bank_account;  
    int balance;  
};
```

```
struct student{  
    char *name;  
    int ID;  
    struct financial *financial;  
    struct academic *academic;  
};
```

```
struct student mike = {  
    .name="Mike", .ID=1  
};  
struct financial example =  
{1234567, 2000};
```

```
mike.financial = example;
```



Structs within structs

It is more common for a struct to contain a *pointer* to the other structs.

```
struct financial {  
    long bank_account;  
    int balance;  
};  
  
struct student{  
    char *name;  
    int ID;  
    struct financial *financial;  
    struct academic *academic;  
};
```

```
struct student mike = {  
    .name="Mike", .ID=1  
};  
struct financial example =  
{1234567, 2000};  
  
mike.financial = &example;
```

The **financial** member of **struct student** is a **pointer** so it must be assigned an **address**.

Structs within structs

It is more common for a struct to contain a *pointer* to the other structs.

```
struct financial {  
    long bank_account;  
    int balance;  
};  
  
struct student{  
    char *name;  
    int ID;  
    struct financial *financial;  
    struct academic *academic;  
};
```

```
struct student mike = {  
    .name="Mike", .ID=1  
};  
struct financial example =  
{1234567, 2000};  
  
mike.financial = &example;  
  
printf("Balance is %d\n",  
    mike.financial.balance);
```



Structs within structs

It is more common for a struct to contain a *pointer* to the other structs.

```
struct financial {  
    long bank_account;  
    int balance;  
};  
  
struct student{  
    char *name;  
    int ID;  
    struct financial *financial;  
    struct academic *academic;  
};
```

```
struct student mike = {  
    .name="Mike", .ID=1  
};  
struct financial example =  
{1234567, 2000};  
  
mike.financial = &example;  
  
printf("Balance is %d\n",  
    mike.financial->balance);
```

Bit fields

Bitwise operations are complex.

Another mechanism is bit fields in structs:

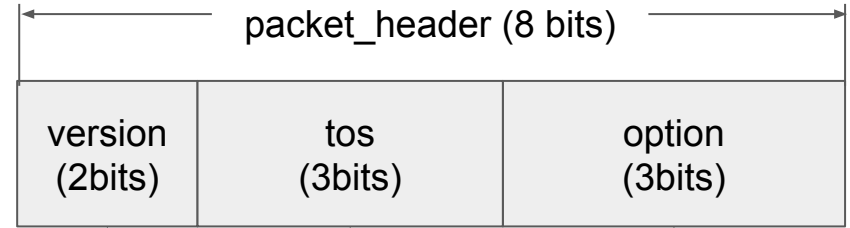
```
struct packet_header{
```

```
    unsigned version: 2;
```

```
    unsigned tos: 3;
```

```
    unsigned option: 3;
```

```
} hdr;
```

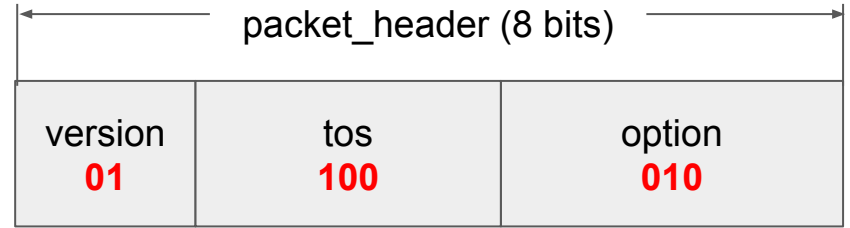


Bit fields

Bitwise operations are complex.

Another mechanism is bit fields in structs:

```
struct packet_header{  
    unsigned version: 2;  
  
    unsigned tos: 3;  
  
    unsigned option: 3;  
} hdr;  
hdr.version=1; hdr.tos=4; hdr.option=2;
```



break

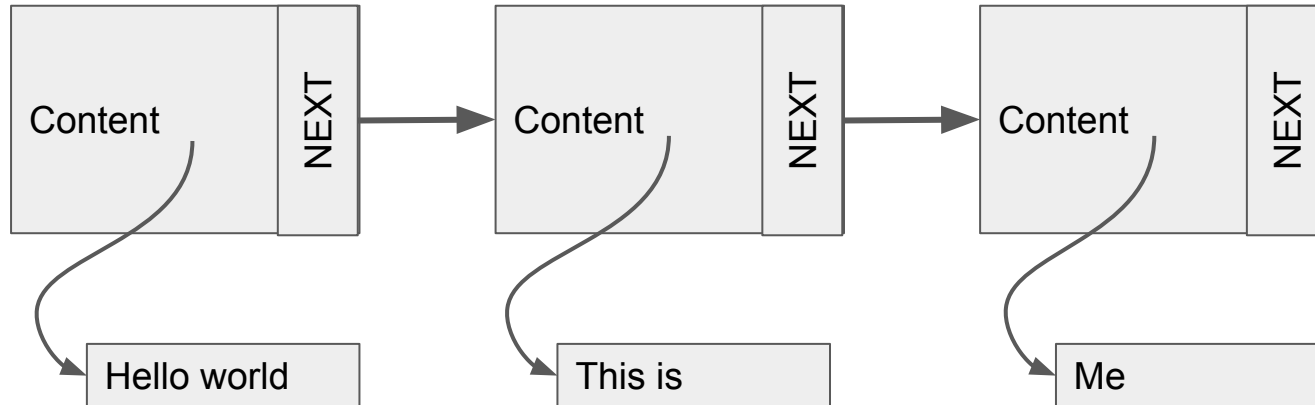
Self-referential structs

```
struct staff{  
    char *name;  
    int ID;  
    struct staff *manager;  
};
```

```
struct staff montyburns;  
struct staff homersimpson =  
    {"Homer", 666, &montyburns};
```

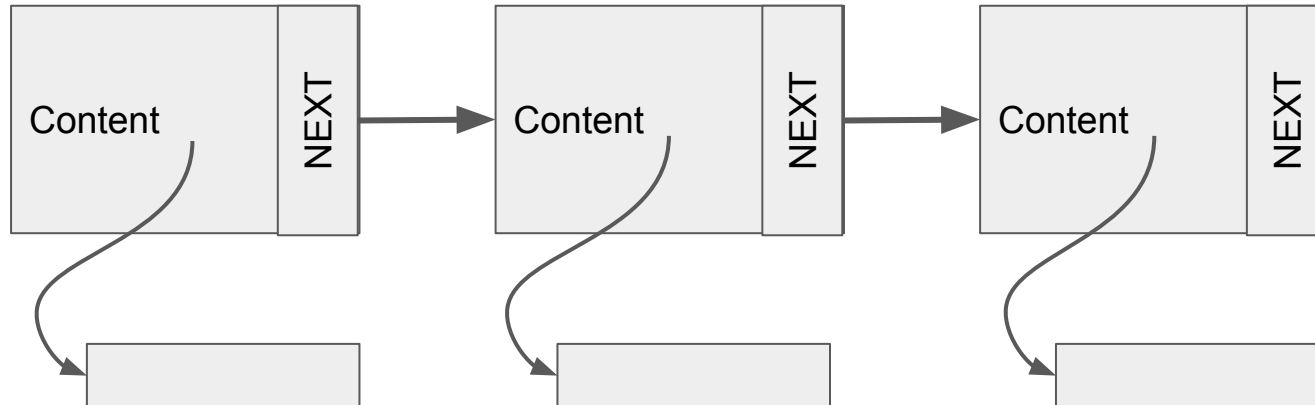
This is legal syntax.

Self-referential structs



Self referential structs: **linked lists**

```
struct node{  
    void *contents;  
    struct node *next;  
};
```

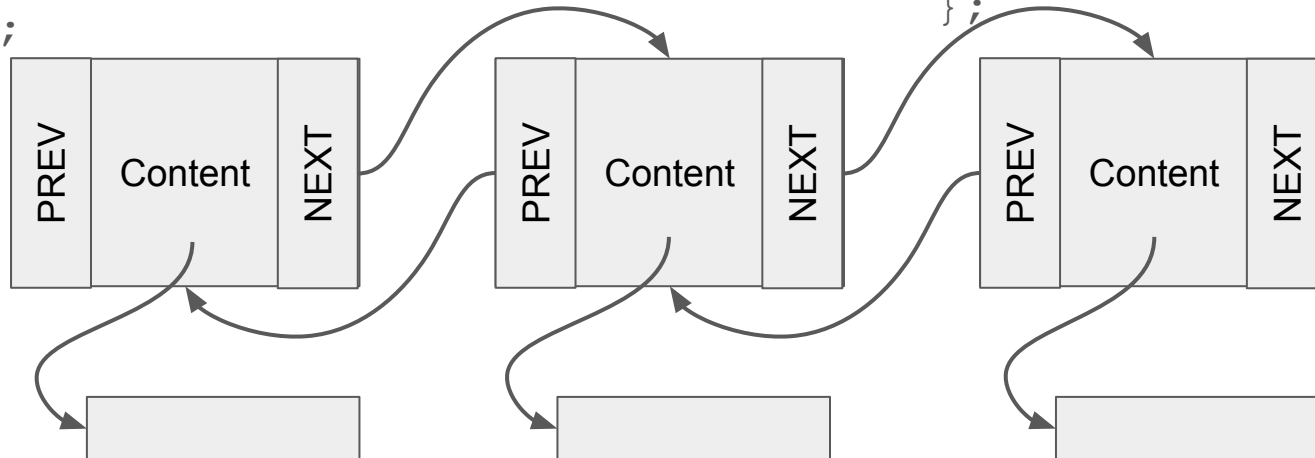


Linked lists

Linked lists

Singly-linked list

```
struct node{  
    void *contents;  
    struct node *next;  
};
```



Doubly-linked list

```
struct node{  
    void *contents;  
    struct node *prev;  
    struct node *next;  
};
```

Lists vs arrays

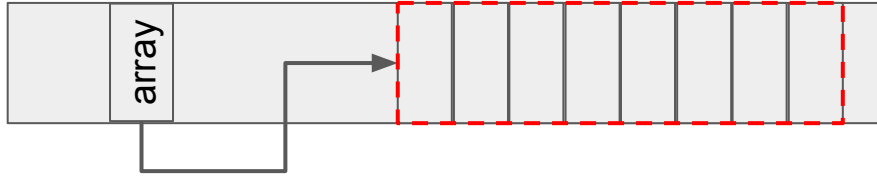
Array

- Contiguous list of values in memory
- Allocated all at once
- Can only modify the available elements
- Cannot add or remove elements

List

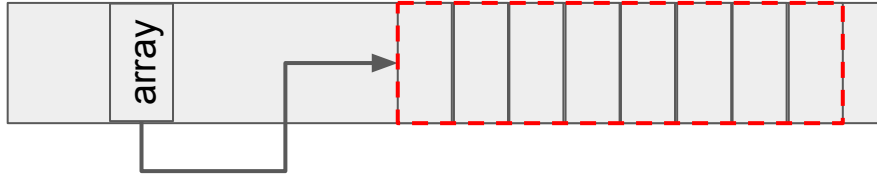
- Allocated on demand, does not need to be contiguous memory region
- Can add, insert, and remove nodes

Lists vs arrays - diagram



```
int *array =  
malloc(sizeof(int)*number);  
array[3] = 20;  
for (i=0;i<number;i++)  
    printf("%d ", array[i]);
```

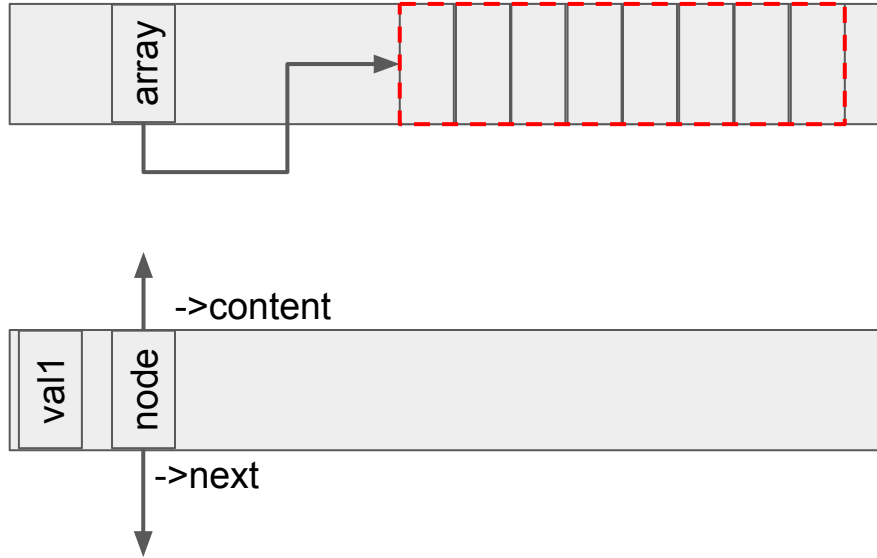
Lists vs arrays - diagram



```
int *array =  
malloc(sizeof(int)*number);  
array[3] = 20;  
for (i=0;i<number;i++)  
    printf("%d ", array[i]);  
  
int *val1 = malloc(sizeof(int));
```

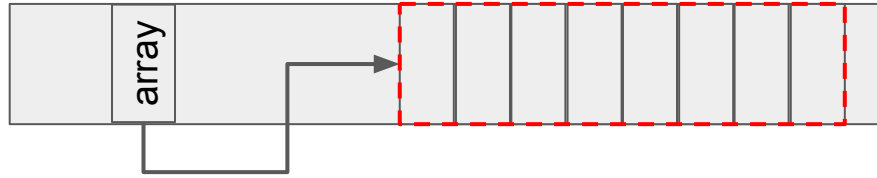


Lists vs arrays - diagram

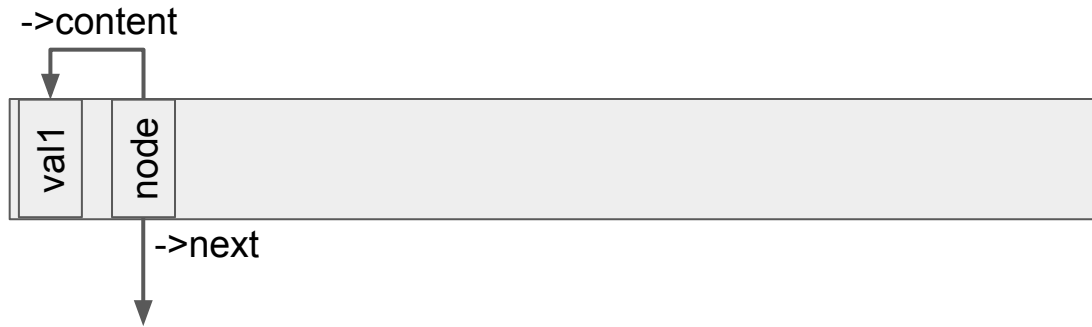


```
int *array =  
malloc(sizeof(int)*number);  
array[3] = 20;  
for (i=0;i<number;i++)  
    printf("%d ", array[i]);  
  
    int *val1 = malloc(sizeof(int));  
    ...  
    n1 = init_list(val1);
```

Lists vs arrays - diagram

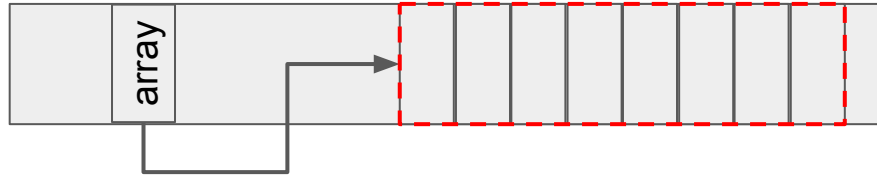


```
int *array =  
malloc(sizeof(int)*number);  
array[3] = 20;  
for (i=0;i<number;i++)  
    printf("%d ", array[i]);
```



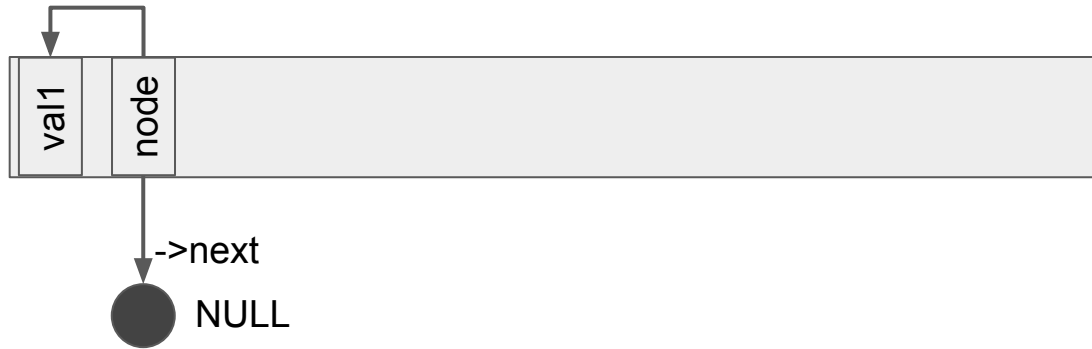
```
int *val1 = malloc(sizeof(int));  
...  
n1 = init_list(val1);
```

Lists vs arrays - diagram



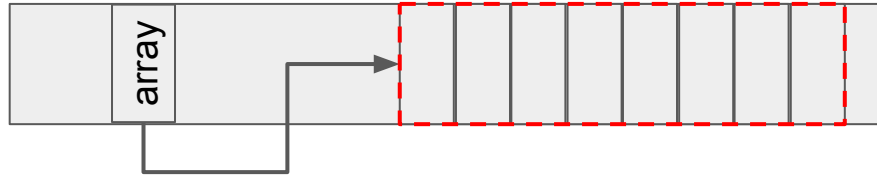
```
int *array =  
malloc(sizeof(int)*number);  
array[3] = 20;  
for (i=0;i<number;i++)  
    printf("%d ", array[i]);
```

->content



```
int *val1 = malloc(sizeof(int));  
...  
n1 = init_list(val1);
```

Lists vs arrays - diagram



```
int *array =  
malloc(sizeof(int)*number);  
array[3] = 20;  
for (i=0;i<number;i++)  
    printf("%d ", array[i]);
```

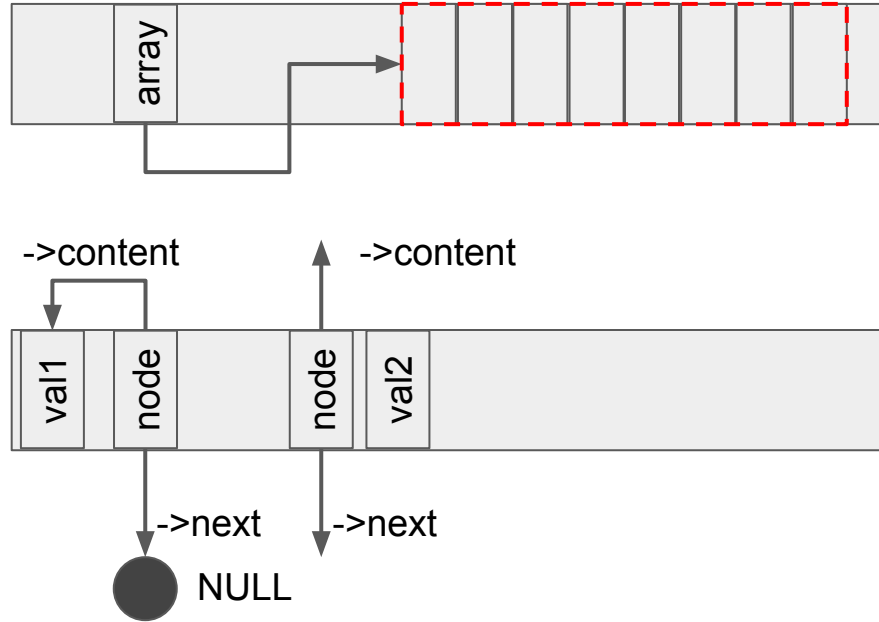
->content



->next
NULL

```
int *val1 = malloc(sizeof(int));  
...  
n1 = init_list(val1);  
...  
int *val2 = malloc(sizeof(int));  
...
```

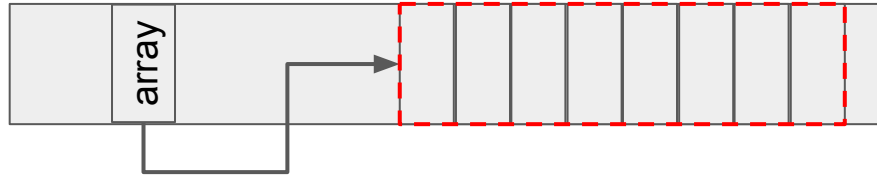
Lists vs arrays - diagram



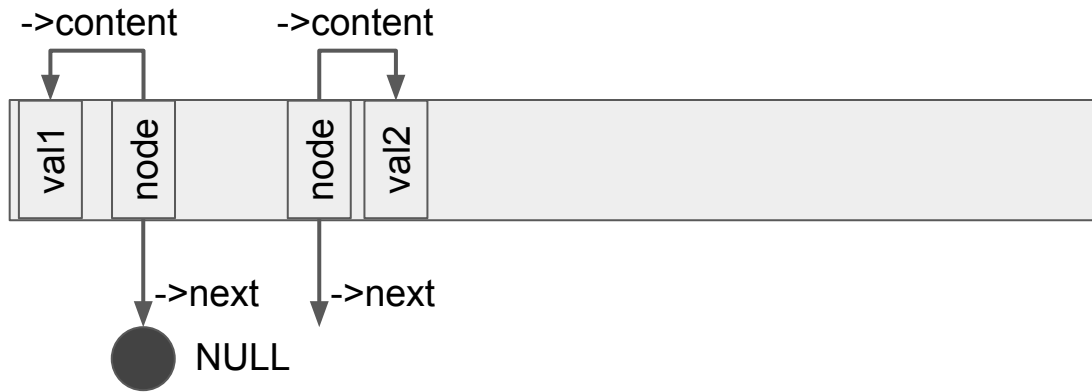
```
int *array =  
malloc(sizeof(int)*number);  
array[3] = 20;  
for (i=0;i<number;i++)  
    printf("%d ", array[i]);
```

```
int *val1 = malloc(sizeof(int));  
...  
n1 = init_list(val1);  
...  
int *val2 = malloc(sizeof(int));  
...  
n2 = add_node(n1, val2);
```

Lists vs arrays - diagram

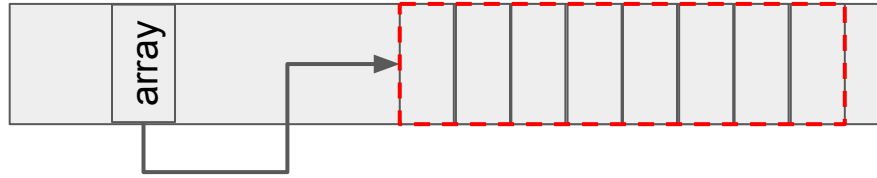


```
int *array =  
malloc(sizeof(int)*number);  
array[3] = 20;  
for (i=0;i<number;i++)  
    printf("%d ", array[i]);
```

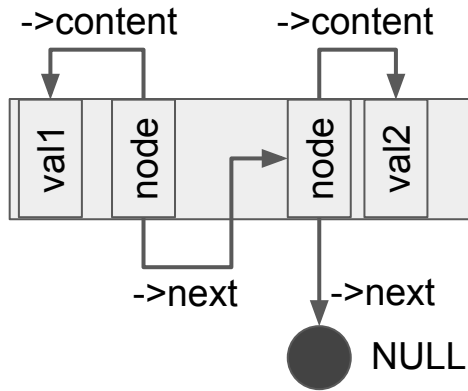


```
int *val1 = malloc(sizeof(int));  
...  
n1 = init_list(val1);  
...  
int *val2 = malloc(sizeof(int));  
...  
n2 = add_node(n1, val2);
```


Lists vs arrays - diagram

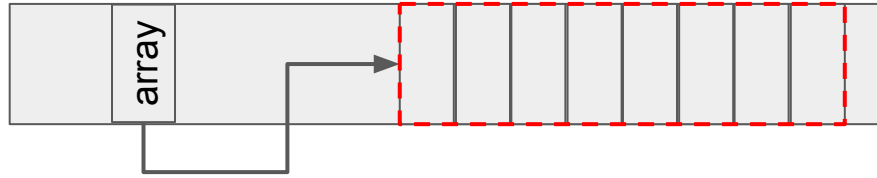


```
int *array =  
malloc(sizeof(int)*number);  
array[3] = 20;  
for (i=0;i<number;i++)  
    printf("%d ", array[i]);
```

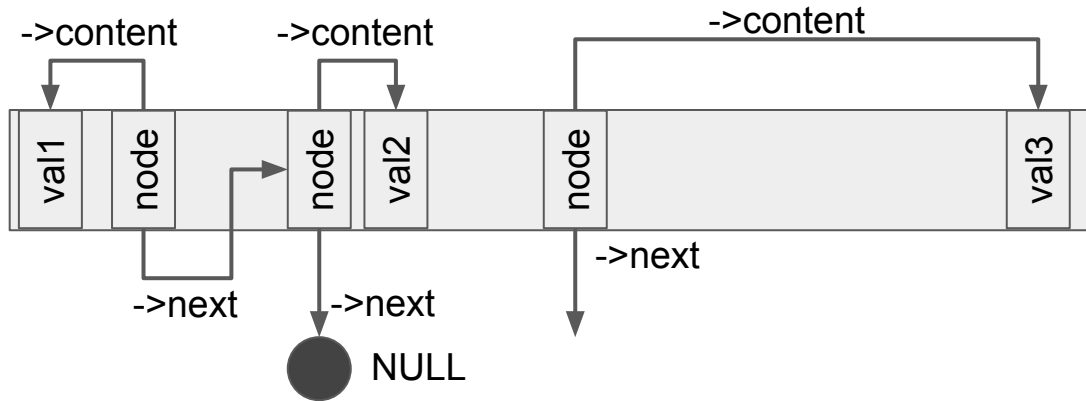


```
int *val1 = malloc(sizeof(int));  
...  
n1 = init_list(val1);  
...  
int *val2 = malloc(sizeof(int));  
...  
n2 = add_node(n1, val2);
```

Lists vs arrays - diagram

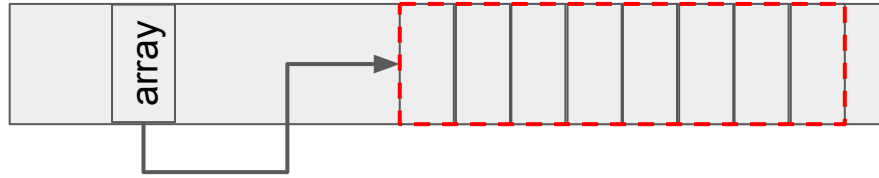


```
int *array =  
malloc(sizeof(int)*number);  
array[3] = 20;  
for (i=0;i<number;i++)  
    printf("%d ", array[i]);
```

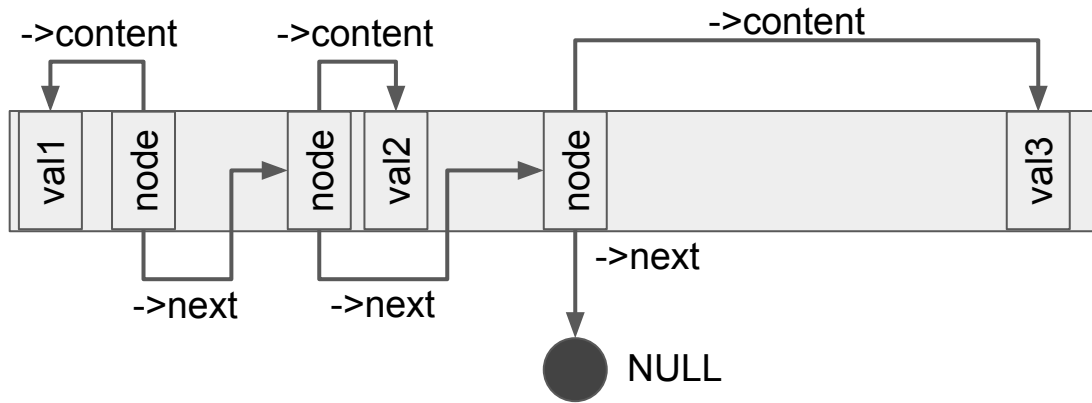


```
int *val1 = malloc(sizeof(int));  
...  
n1 = init_list(val1);  
...  
n2 = add_node(n1, val2);  
...  
n3 = add_node(n1, val3);
```

Lists vs arrays - diagram

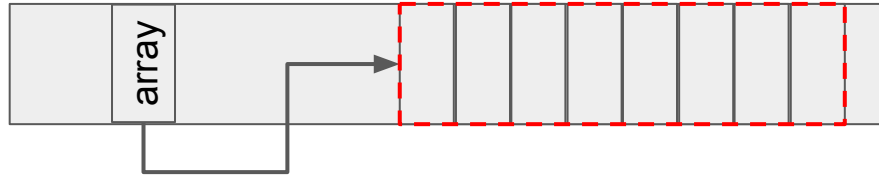


```
int *array =  
malloc(sizeof(int)*number);  
array[3] = 20;  
for (i=0;i<number;i++)  
    printf("%d ", array[i]);
```

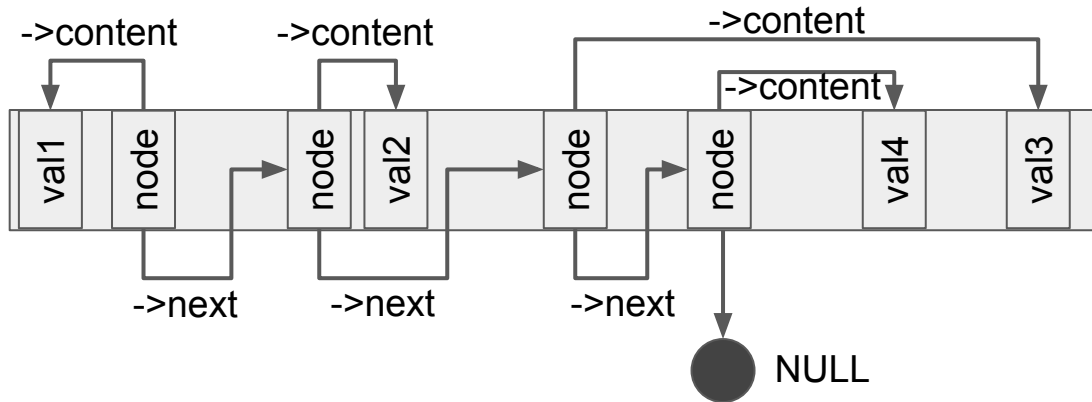


```
int *val1 = malloc(sizeof(int));  
...  
n1 = init_list(val1);  
...  
n2 = add_node(n1, val2);  
...  
n3 = add_node(n1, val3);
```

Lists vs arrays - diagram



```
int *array =  
malloc(sizeof(int)*number);  
array[3] = 20;  
for (i=0;i<number;i++)  
    printf("%d ", array[i]);
```



```
int *val1 = malloc(sizeof(int));  
...  
n1 = init_list(val1);  
...  
n2 = add_node(n1, val2);  
...  
n3 = add_node(n1, val3);  
...  
n4 = add_node(n1, val4);
```

Example: list of students

With array we need to allocate the full list of students at the start

- This means that we have to know the number of students

```
struct student {  
    char *name;  
    int id;  
};  
struct student students[20];
```

```
//OR, dynamically  
struct student *students;  
int num_students;  
scanf("%d", &num_students);  
students = malloc(sizeof(struct student) * num_students);
```

Example: list of students

With array we need to allocate the full list of students at the start

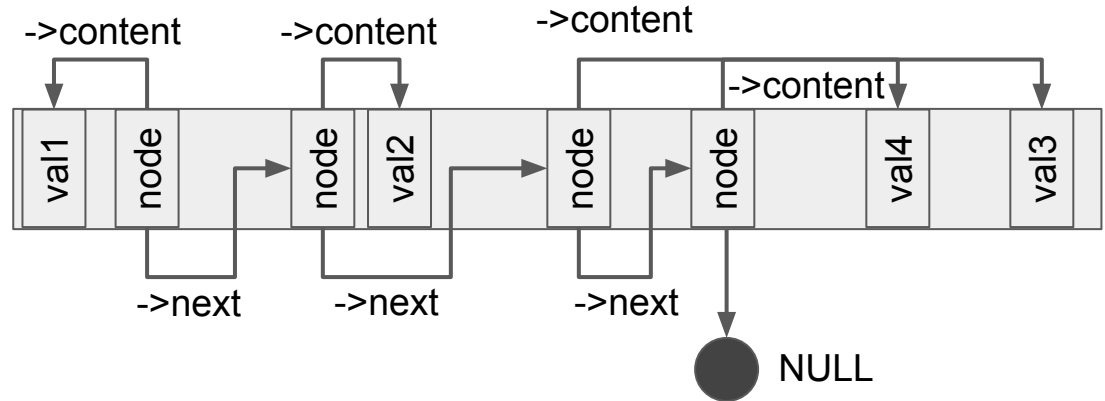
- This means that we have to know the number of students

```
struct student {  
    char *name;  
    int id;  
};  
struct list *students = init_list();  
while (1) {  
    char *name = malloc(20);  
    scanf("%s", name);  
    struct student *s = malloc(sizeof(struct student));  
    s->name = name; s->id = id++;  
    add_node(students, s);...
```

Linked list: searching the list

- Go through the list nodes using the **next** field in the node structure
- Use a pointer to go from node to node
- Stop when we hit “NULL” (end of the list)

Input: `list head (struct list *head); value (void *value)`

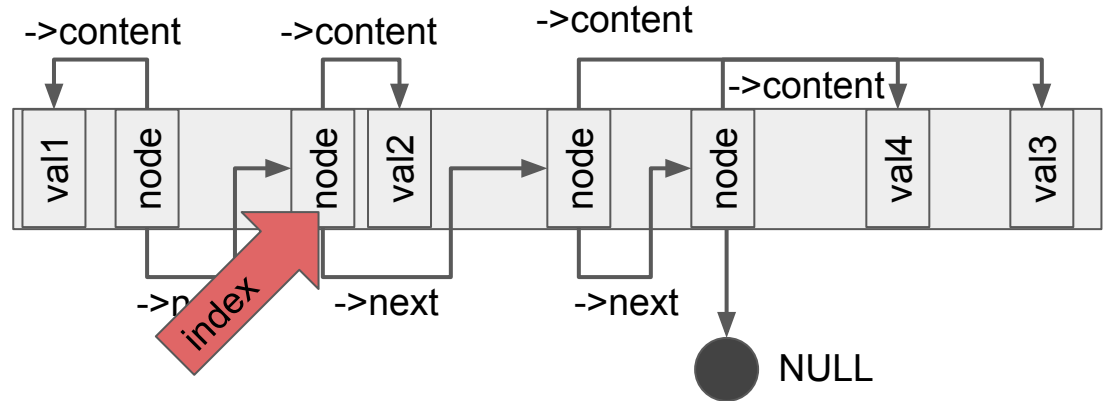


Linked list: searching the list

- Go through the list nodes using the **next** field in the node structure
- Use a pointer to go from node to node
- Stop when we hit “NULL” (end of the list)

Input: list head (struct list *head); value (void *value)

```
struct list *index;  
index = head;  
  
while (index != NULL) {  
    if (index->content ==  
        val) {  
        break;  
    } else {  
        index = index->next;  
    }  
}
```

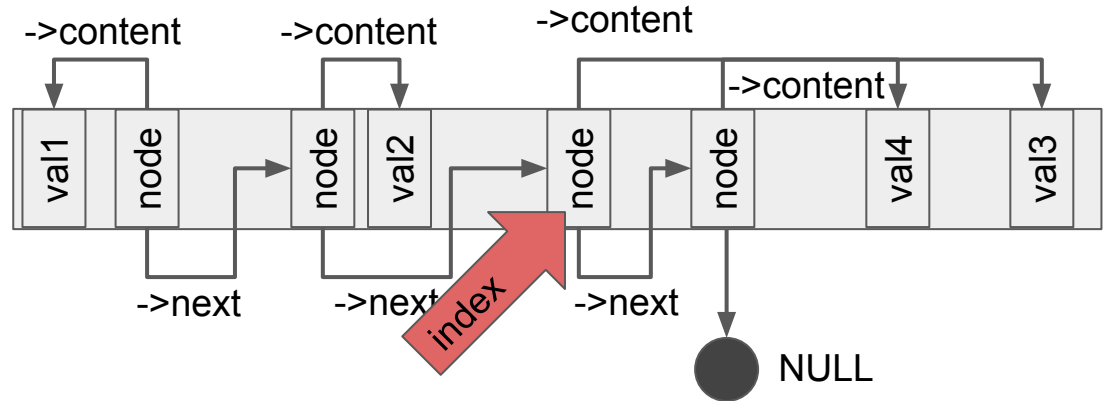


Linked list: searching the list

- Go through the list nodes using the **next** field in the node structure
- Use a pointer to go from node to node
- Stop when we hit “NULL” (end of the list)

Input: `list head (struct list *head); value (void *value)`

```
struct list *index;  
index = head;  
  
while (index != NULL) {  
    if (index->content ==  
        val) {  
        break;  
    } else {  
        index = index->next;  
    }  
}
```

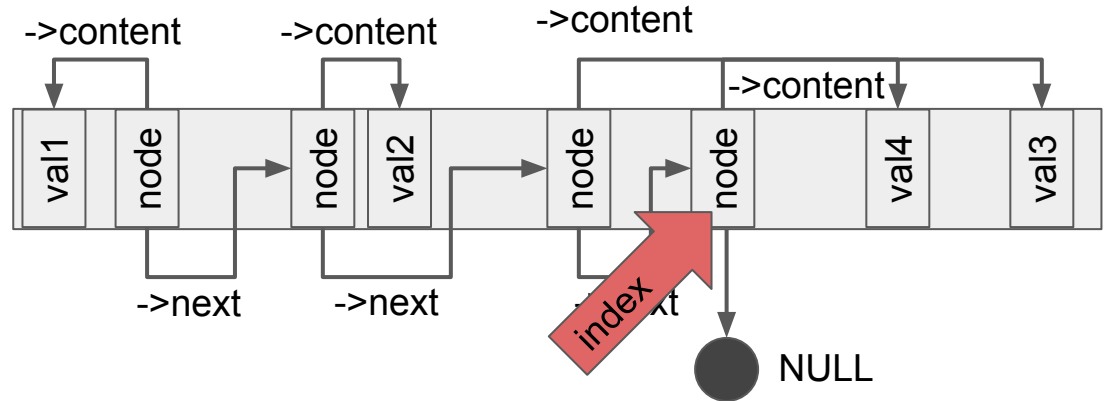


Linked list: searching the list

- Go through the list nodes using the **next** field in the node structure
- Use a pointer to go from node to node
- Stop when we hit “NULL” (end of the list)

Input: list head (struct list *head); value (void *value)

```
struct list *index;  
index = head;  
  
while (index != NULL) {  
    if (index->content ==  
        val) {  
        break;  
    } else {  
        index = index->next;  
    }  
}
```

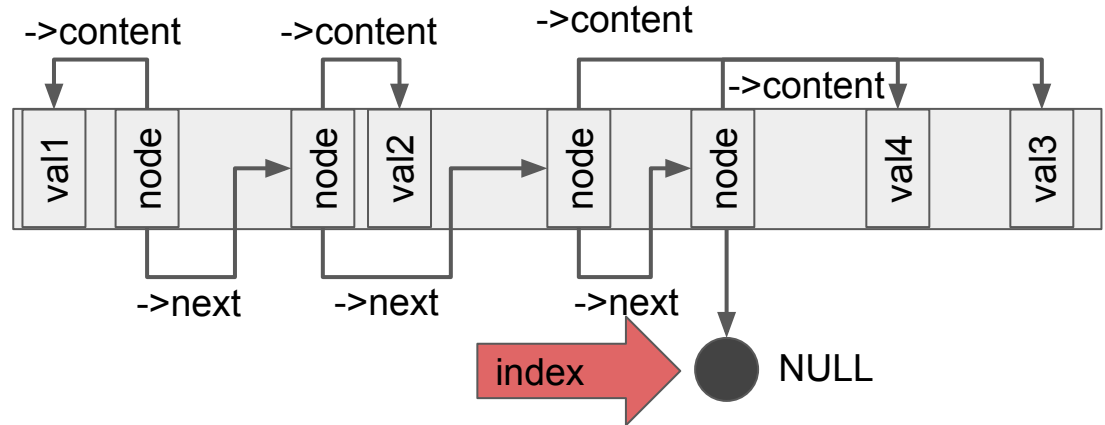


Linked list: searching the list

- Go through the list nodes using the **next** field in the node structure
- Use a pointer to go from node to node
- Stop when we hit “NULL” (end of the list)

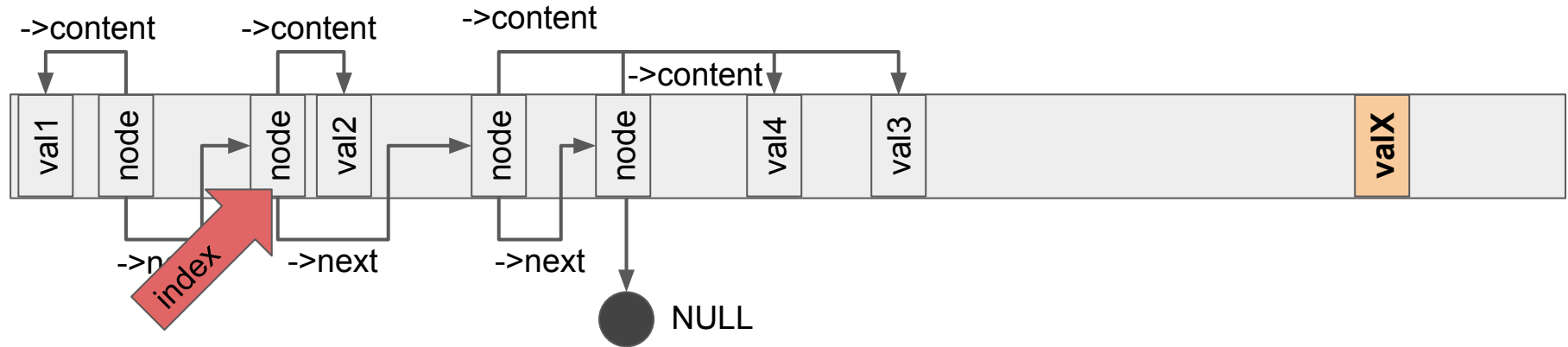
Input: list head (struct list *head); value (void *value)

```
struct list *index;  
index = head;  
  
while (index != NULL) {  
    if (index->content ==  
        val) {  
        break;  
    } else {  
        index = index->next;  
    }  
}
```



Linked list: adding/inserting nodes

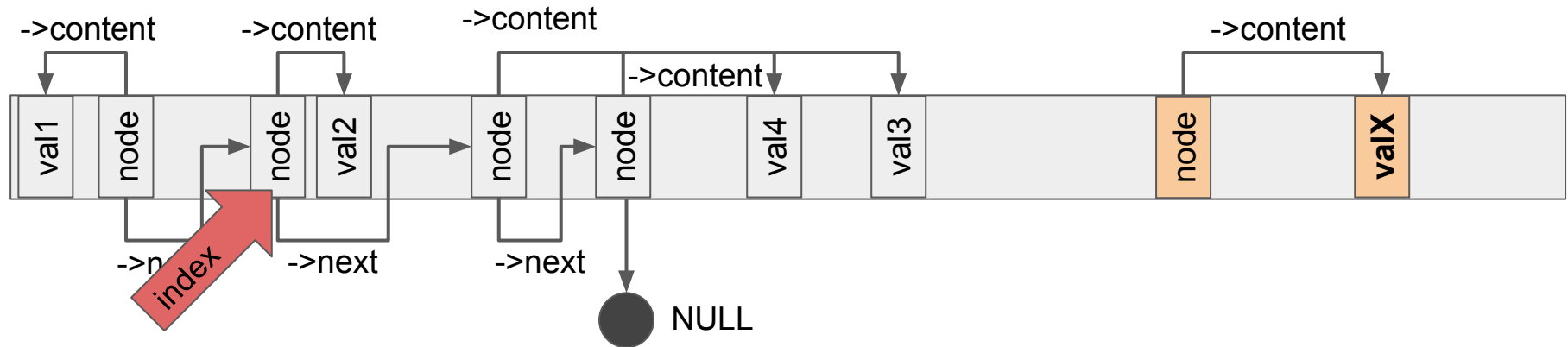
Input: node after which to add (struct list *index); value



Linked list: adding/inserting nodes

Input: node after which to add (`struct list *index`); value

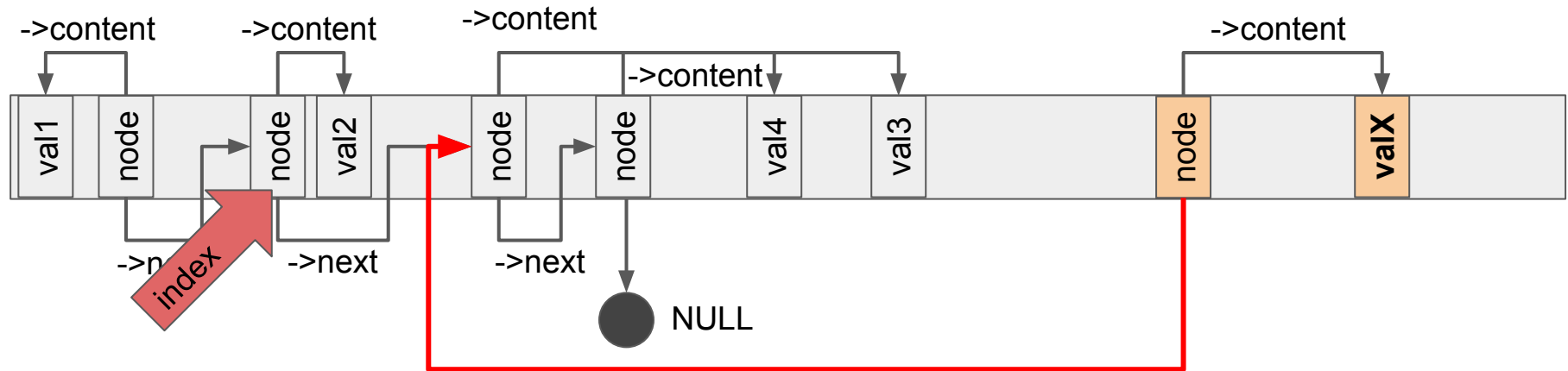
- Allocate memory for the new node



Linked list: adding/inserting nodes

Input: node after which to add (struct list *index); value

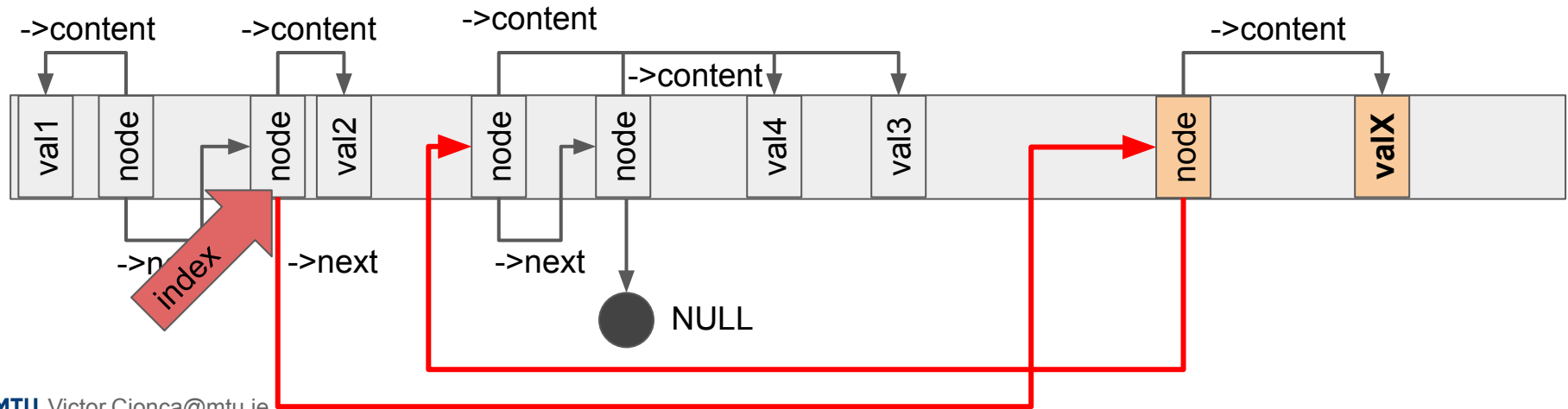
- Allocate memory for the new node
 - Rewire the **next** connections. →
1. Connect new node to next node



Linked list: adding/inserting nodes

Input: node after which to add (struct list *index); value

- Allocate memory for the new node
 - Rewire the **next** connections. →
1. Connect new node to next node
 2. Connect the index to the new node



Linked list: adding/inserting nodes

Special case when adding before the list head

In that case it is called “**pushing**” into the list

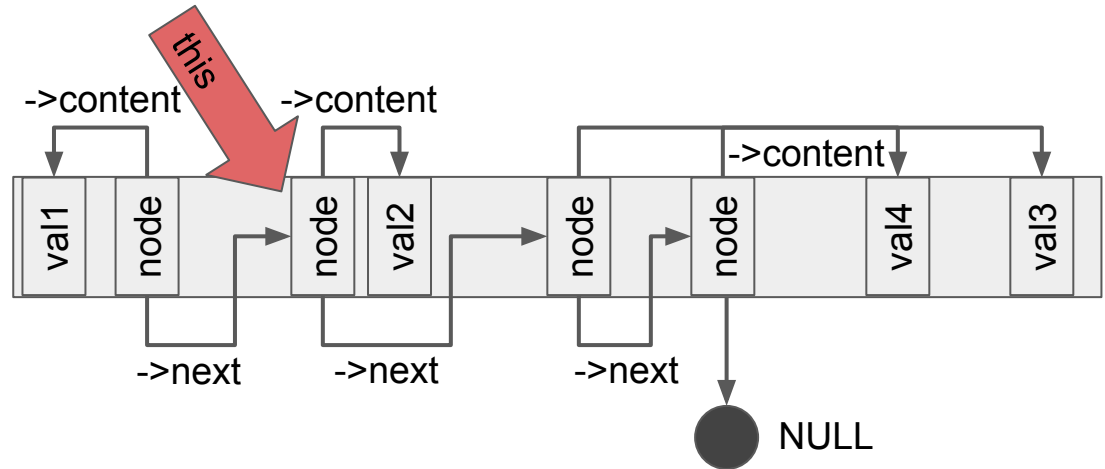
There is no “previous” node, simply connect the new node to the list head

Replace the list head with the new node.

Linked list: removing nodes

For singly linked lists:

- Can directly remove “next” node
- Can remove “this” node but needs to first search through the list until `index->next == this`

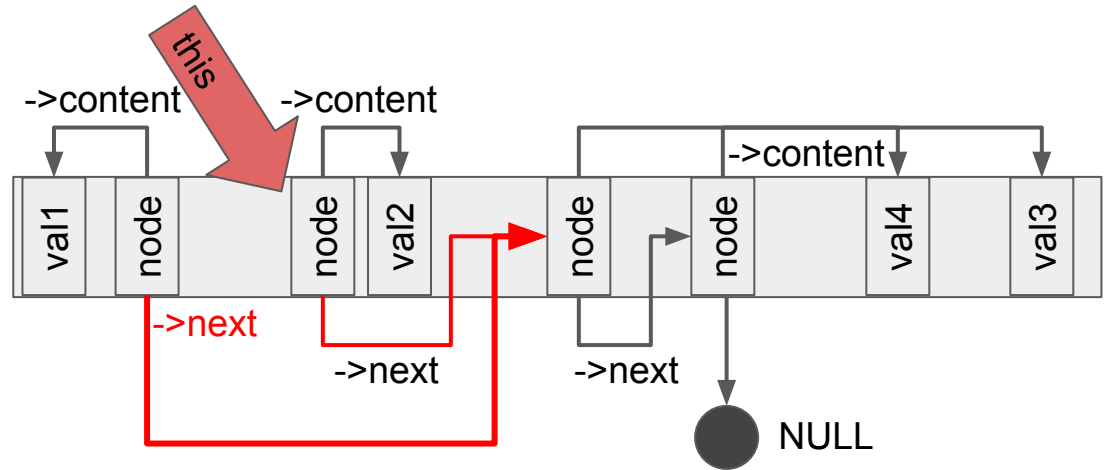


Linked list: removing nodes

For singly linked lists:

- Can directly remove “next” node
- Can remove “this” node but needs to first search through the list until `index->next == this`

1. Rewire previous node to node after “this”

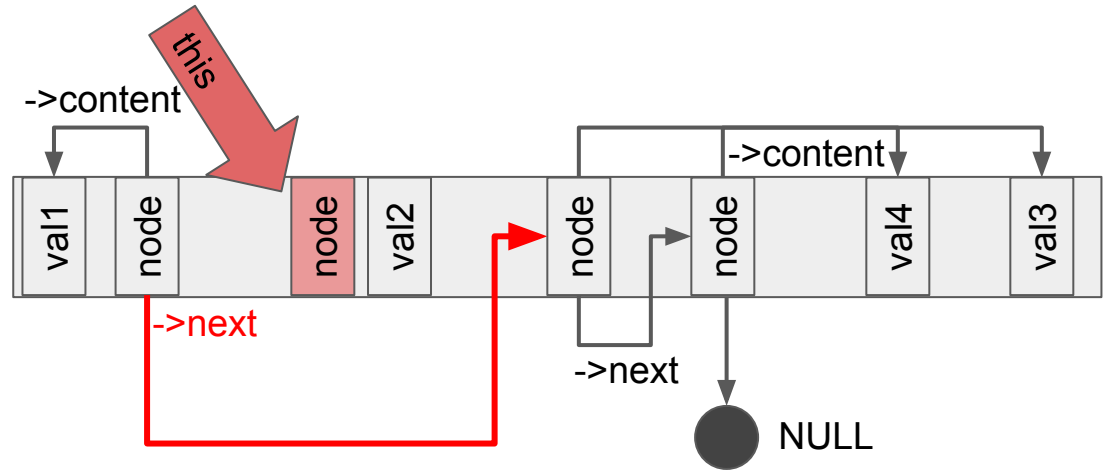


Linked list: removing nodes

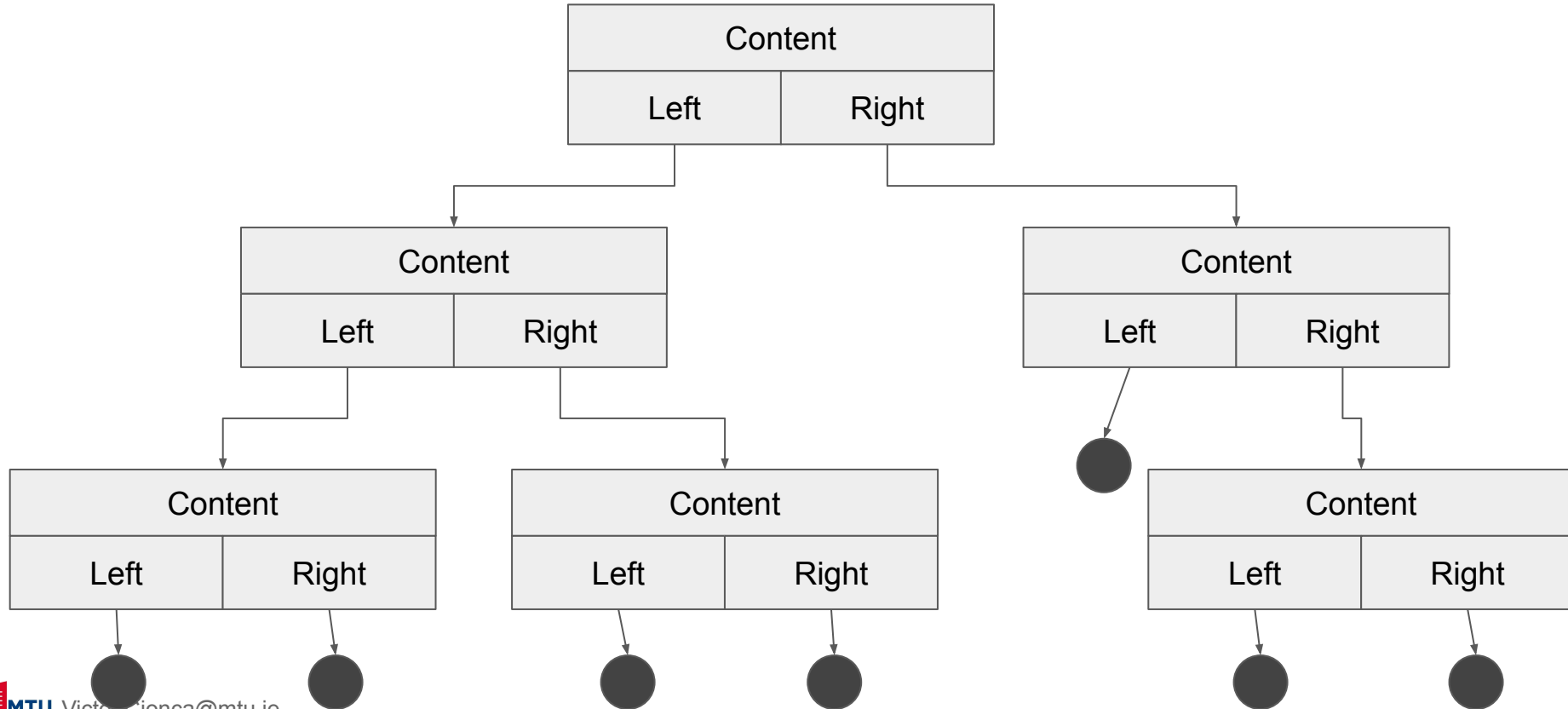
For singly linked lists:

- Can directly remove “next” node
- Can remove “this” node but needs to first search through the list until $\text{index} \rightarrow \text{next} == \text{this}$

1. Rewire previous node to node after “this”
2. Free the memory allocated to “this”



Another self-referential structure: trees



Enums

Similar to Java **enums**

Quickly define a set of constant values

```
enum Month {JAN, FEB, MAR, ...};
```

In the program **enum** constants can be used like regular **int** constants

```
int month = JAN;
```

Value of enum constants

```
enum Month {JAN, FEB, MAR, ...};  
printf("%d\n", JAN); // what will this print?
```

1. Implicit initialisation of constants values
 - a. Used when we define the enum as above
 - b. Constants are assigned values incrementally starting at 0
 - c. JAN=0, FEB=1, MAR=2...

Value of enum constants

Explicit initialisation:

```
enum size {  
    SIZE_CHAR = 1,  
    SIZE_SHORT = 2,  
    SIZE_INT = 4,  
};
```


Value of enum constants

Mixed initialisation:

```
enum code {  
    VAL_A, VAL_B, VAL_C,  
    VAL_D=10, VAL_E,  
    VAL_F=20, VAL_G,  
};  
printf("%d %d %d\n", VAL_A, VAL_B, VAL_C);  
printf("%d %d\n", VAL_E, VAL_G);
```

In mixed initialisers
the values are assigned incremental integers
from the last assigned constant, or zero.

Using enums

Most times it is only the enum constants that are used in the program.

Enum constants are useful in **switch** statements.

Pseudo-polymorphism in C

Network software

- Must represent the address of a device in a generic way
- Support any type of device: IPv4 (32bit), IPv6 (128bit), WiFi, TCP ports, etc

In Java:

- Abstract NetworkAddress class
- Subclasses for each device type

Unions

```
union Address {  
    int tcp_port;  
    char ipv4[4];  
    char ipv6[16];  
    char wifi[6];  
};  
union Address addr;  
addr.tcp_port = 7;  
printf("%d\n", addr.tcp_port);  
init(&addr.ipv4, 192,168,1,10);  
printf("%d.%d.%d.%d\n", addr.ipv4[0],...);
```

Unions

```
union Address {  
    int tcp_port;  
    char ipv4[4];  
    char ipv6[16];  
    char wifi[6];  
};  
union Address addr;  
addr.tcp_port = 7;  
printf("%d\n", addr.tcp_port);  
init(&addr.ipv4, 192,168,1,10);  
printf("%d.%d.%d.%d\n", addr.ipv4[0],...);
```

```
> ./test  
7  
192.168.1.10
```

Unions

```
union Address {  
    int tcp_port;  
    char ipv4[4];  
    char ipv6[16];  
    char wifi[6];  
};  
union Address addr;  
addr.tcp_port = 7;  
printf("%d\n", addr.tcp_port);  
init(&addr.ipv4, 192,168,1,10);  
printf("%d.%d.%d.%d\n", addr.ipv4[0],...);  
printf("%d\n", addr.tcp_port);
```

This looks and feels just like structs.
However...

```
> ./test  
7  
192.168.1.10
```

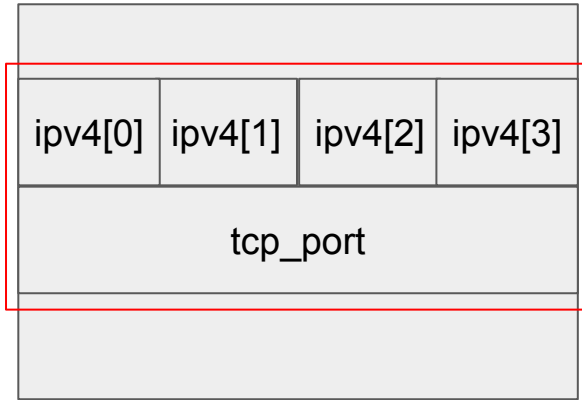
Unions

```
union Address {  
    int tcp_port;  
    char ipv4[4];  
    char ipv6[16];  
    char wifi[6];  
};  
union Address addr;  
addr.tcp_port = 7;  
printf("%d\n", addr.tcp_port);  
init(&addr.ipv4, 192,168,1,10);  
printf("%d.%d.%d.%d\n", addr.ipv4[0],...);  
printf("%d\n", addr.tcp_port);
```

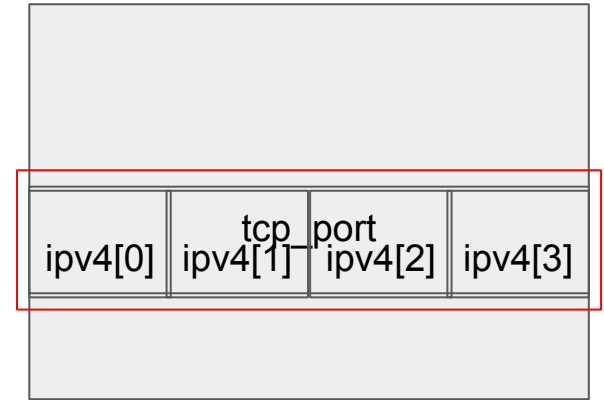
This looks and feels just like structs.
However...

```
> ./test  
7  
192.168.1.10  
192
```

Unions vs Structs



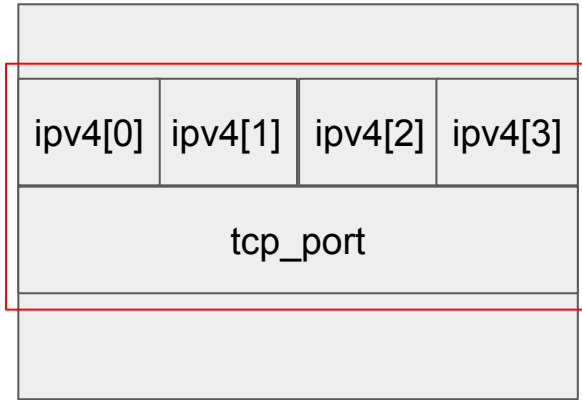
```
struct Address {  
    int tcp_port;  
    char ipv4[4];  
};
```



```
union Address {  
    int tcp_port;  
    char ipv4[4];  
};
```

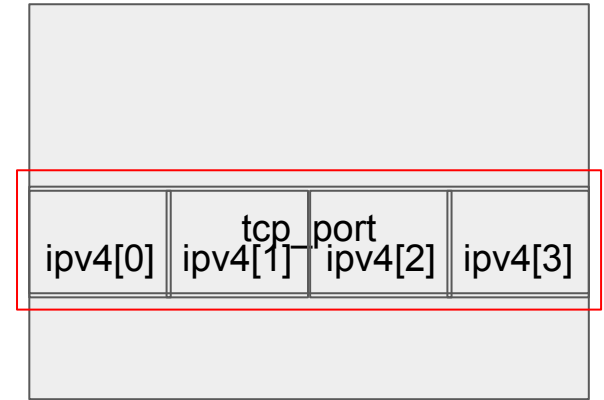

Unions vs Structs

In a struct its members occupy consecutive areas in memory (like in an array).



```
struct Address {  
    int tcp_port;  
    char ipv4[4];  
};
```

In a union all members start at the same location.



```
union Address {  
    int tcp_port;  
    char ipv4[4];  
};
```

Tagged unions (pseudo-polymorphism)

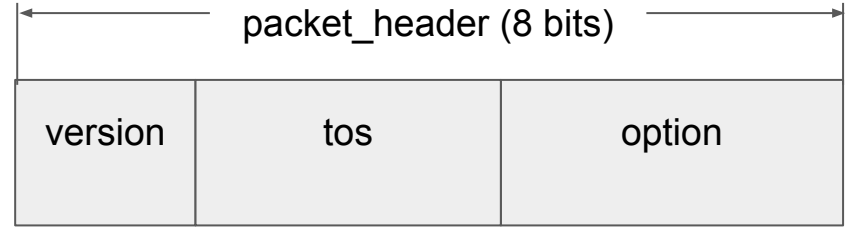
```
enum AddressType={TCP, IP4};
struct Address {
    enum AddressType type;
    union Address {
        int tcp_port;
        char ipv4[4];
    };
};

union Address addr;
switch (addr.type){
    case TCP:
        connect_tcp(addr.tcp_port);
        break;
    case IP4:
        connect_ip4(addr.ipv4);
        break;
}
```

Bit fields: assignment

Assigning to bit fields requires a union

```
union pkt_hdr{
    char byte;
    struct {
        unsigned version: 2;
        unsigned tos: 3;
        unsigned option: 3;
    } fields;
}
```



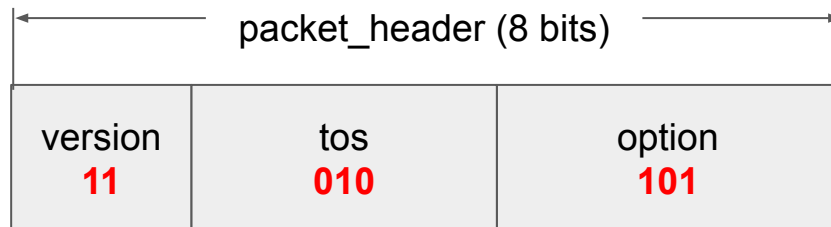
```
void main(void) {
    union pkt_hdr p;
    p.byte = 0xD5; // binary 11010101

    printf("Version: %d\n",
           p.fields.version);
    printf("TOS: %d\n",
           p.fields.tos);
    printf("Option: %d\n",
           p.fields.option);
}
```

Bit fields: assignment

Assigning to bit fields requires a union

```
union pkt_hdr{
    char byte;
    struct {
        unsigned version: 2;
        unsigned tos: 3;
        unsigned option: 3;
    } fields;
}
```



```
void main(void){
    union pkt_hdr p;
    p.byte = 0xD5; // binary 11010101

    printf("Version: %d\n",
           p.fields.version);
    printf("TOS: %d\n",
           p.fields.tos);
    printf("Option: %d\n",
           p.fields.option);
}
```

Typedef

Functionality for creating alias data types.

Syntax: `typedef <original type> <alias>;`

The alias can then be used as a valid data type.

Typedef examples

The following are common in embedded systems:

```
typedef unsigned char uint8_t;  
typedef unsigned short uint16_t;  
typedef unsigned int uint32_t;  
uint8_t packet_hdr[10]; // array of 8 bytes
```

It is common to alias struct types for conciseness:

```
typedef struct{ char name[20]; int id; } student_t;
```